

JOBINTECH

RAPPORT DE MINI-PROJETS

Réalisé Par :

MOHAMED FEJJARY

BRAHIM DARGUI

SOMMAIRE

Introduction	1
Projet 1 - Système de point de vente (Café)	2
Projet 2 - Analyse de sentiment détaillée	3
Projet 3 - Prototype d'inventaire (Librairie)	4
Projet 4 - Analyseur de logs amélioré	5
Projet 5 - Outil de budgétisation personnel	6
Conclusion & remerciements	7

INTRODUCTION

Ce document présente une analyse synthétique mais professionnelle de cinq mini-projets de développement. L'objectif est de mettre en évidence les besoins métiers, les compétences techniques mobilisées ainsi que la valeur ajoutée de chaque solution. Ces projets illustrent différentes facettes de la programmation appliquée à des cas concrets tels que la gestion commerciale, l'analyse de données, la tenue d'inventaire, le suivi financier ou encore l'analyse de logs.

01

Objectif : Développer une fonction flexible de génération de reçus intégrant réductions et programme de fidélité.

Exigences principales :

- Gestion d'un sous-total, réductions, taxe et points fidélité.
- Codes promotionnels : réduction en pourcentage ou montant fixe.
- Reçu formaté de manière professionnelle.



Approche : Encapsulation de la logique dans une fonction unique (generer_recu). Utilisation de paramètres optionnels pour la réduction et calcul conditionnel. Retour du total final et impression d'un reçu lisible.

Valeur ajoutée : Améliore l'expérience client et la flexibilité de gestion commerciale.


```

#POS pour une coffee shop local

#La commande client
commande_client = [
    {'nom': 'Latte', 'prix': 4.25, 'quantite': 2},
    {'nom': 'Muffin', 'prix': 3.25, 'quantite': 1}
]

#La fonction pour generer le reçu de la commande
def generer_recu(liste_commande, taux_taxe, code_reduction=None):
    sous_total = 0.0
    for commande in liste_commande:
        prix = commande['prix']
        quantite = commande['quantite']
        sous_total += prix * quantite

    #Appliquer la reduction si existe
    montant_reduction = 0.0
    rate_reduction = None
    if code_reduction:
        if code_reduction == 'SAVE10': #code 'SAVE10' pour reduire 10%
            montant_reduction = sous_total * 0.10
            rate_reduction = '10%'
        elif code_reduction == '50FF': #code '50FF' pour -5€
            montant_reduction = 5.0
            rate_reduction = '5€'
            if sous_total < montant_reduction: #si le montant réduction > sous-total
                montant_reduction = sous_total
        elif code_reduction == '' or code_reduction is None: #code vide ou non déclaré
            montant_reduction = 0.0
        else: #si un code inconnu
            montant_reduction = 0.0

    #Calculs
    montant_avec_reduction = sous_total - montant_reduction
    montant_taxe = montant_avec_reduction * taux_taxe
    grand_total = montant_avec_reduction + montant_taxe
    points_fedilite = int(grand_total // 5) # 1 point /5€

    #Imprimer le reçu formaté
    print("==== Reçu du Café =====")
    for commande in liste_commande:
        print(f"{commande['nom']}\t\t x {commande['quantite']}\t\t{commande['prix']:.2f}€")
    print('-'*26)
    print(f"Sous_total: \t\t{sous_total:.2f}€")
    print(f"Réduction ({rate_reduction}):\t-{montant_reduction:.2f}€" if montant_reduction != 0 else
    "Réduction:\t\t\t 0.00€")
    print(f"Taxe ({taux_taxe*100:.0f}%):\t\t\t {montant_taxe:.2f}€")
    print(f"Total:\t\t\t\t{grand_total:.2f}€")
    print('=' * 26)
    print(f"Points de fidélité gagnés: {points_fedilite}\n")
    #Return le Grand total
    return round(grand_total, 2)

#Test sans code reduction
Test01 = generer_recu(commande_client, 0.07)

#Test avec code reduction 'SAVE10'
Test02 = generer_recu(commande_client, 0.07, 'SAVE10')

#Test avec code reduction '50FF'
Test03 = generer_recu(commande_client, 0.8, '50FF')

#Test avec code reduction Non défini
Test04 = generer_recu(commande_client, 0.10, 'RED10')

```

Objectif : Identifier les retours clients avec nuances, incluant les cas mixtes.

Exigences principales :

- Système de score (+1/-1) basé sur mots-clés.
- Catégorisation : Positif, Négatif, Mixte, Neutre.
- Extraction et restitution des mots déclencheurs.



Approche : Boucle d'analyse par commentaire, stockage des résultats dans une liste de dictionnaires. Retour d'une structure claire pour générer un rapport détaillé.

Valeur ajoutée : Permet d'obtenir des insights précis sur les avis clients et d'orienter les décisions marketing.

```

#Rapport d'Analyse de Sentiment Détaillé
feedback_clients = [
    "Le service était excellent et la nourriture était super!",
    "J'adore le design, mais j'ai été déçu par l'autonomie de la batterie.",
    "Le café est correct, sans plus.",
]

mots_cles_positifs = ['excellent', 'super', 'adore', 'génial', 'satisfait']
mots_cles_negatifs = ['déçu', 'mauvais', 'terrible', 'horrible', 'problème']

#Fonction analyse feedback
def analyser_feedback(donnees_feedback):
    resultats_analyse = []

    for commentaire in donnees_feedback:
        score = 0
        mots_cles_trouves = []

        # Vérifier les positifs
        for mot in mots_cles_positifs:
            if mot in commentaire.lower():
                score += 1
                mots_cles_trouves.append(mot)

        # Vérifier les négatifs
        for mot in mots_cles_negatifs:
            if mot in commentaire.lower():
                score -= 1
                mots_cles_trouves.append(mot)

        # Catégorisation finale
        if score > 0:
            sentiment_final = "Positif"
        elif score < 0:
            sentiment_final = "Négatif"
        elif score == 0 and mots_cles_trouves:
            sentiment_final = "Mixte"
        else:
            sentiment_final = "Neutre"

        # Construire le résultat pour ce commentaire
        resultat = {
            "commentaire": commentaire,
            "sentiment": sentiment_final,
            "score": score,
            "mots_cles": mots_cles_trouves
        }
        resultats_analyse.append(resultat)

    return resultats_analyse

# Utilisation
liste_retournee = analyser_feedback(feedback_clients)

# Rapport détaillé
print("==== Rapport d'Analyse de Sentiment Détaillé =====\n")
for item in liste_retournee:
    print(f"Commentaire: \"{item['commentaire']}\"")
    print(f"- Sentiment: {item['sentiment']} (Score: {item['score']})")
    print(f"- Mots-clés trouvés: {item['mots_cles']}")
    print(f"{'_' * 48}")

```


Objectif : Rendre le système d'inventaire robuste, ergonomique et riche en fonctionnalités.

Exigences principales :

- Validation stricte des entrées utilisateur.
- Ajout de fonctionnalités : recherche, suppression, rapport de stock faible.
- Affichage structuré et lisible.



Approche : Extension du menu interactif, ajout de nouvelles fonctions, et mise en place de boucles de validation. Utilisation de f-strings pour un affichage aligné.

Valeur ajoutée : Outil pratique, évitant les erreurs fréquentes et facilitant la gestion du stock.

```

#=== Menu Inventaire ===
inventaire = { 'Dune': 15, '1984': 5, 'Fondation': 3 }

#Fonction Affichage de menu
def afficher_menu():
    while True:
        input(f"\nAppuyez sur une touche pour Afficher le Menu Principal.. ")
        print("\n===== Menu Inventaire =====")
        print("1. Voir tout l'inventaire")
        print("2. Ajouter un article")
        print("3. Mettre à jour un article")
        print("4. Supprimer un article")
        print("5. rechercher_article")
        print("6. Rapport de Stock Faible")
        print("7. Quitter")
        print("="*28)
        choix = input(f"* Entrez Votre choix : ").strip()

        if not choix.isdigit():
            print("Entrée invalide ! Veuillez entrer un nombre.")
            continue

        choix = int(choix) #convertir en chiffre entier

        if choix == 1:
            afficher_inventaire()
        elif choix == 2:
            ajouter_article()
        elif choix == 3:
            update_article()
        elif choix == 4:
            supprimer_article()
        elif choix == 5:
            rechercher_article()
        elif choix == 6:
            rapport_stock_faible()
        elif choix == 7:
            #confirmation avant Quitter
            confirmation_quit = input(f"Voulez vous vraiment quitter le programme ? (Y/N) ")
            if confirmation_quit.lower() == "n":
                continue
            elif confirmation_quit.lower() == "y":
                print("Au revoir !")
                break
        else:
            print("Choix invalide, veuillez réessayer.")

#--- Appel au fonctions de menu ---#

#la fonction pour afficher le stock actuel
def afficher_inventaire ():
    print("\n**** Inventaire Actuel ****")
    print("-" * 28)
    if not inventaire:
        print("Inventaire est vide !")
    else:
        print(f" • {'Article':<12} | {'Quantité'}")
        print('-'*28)
        for article, quantite in inventaire.items():
            print(f" • {article:<12} | Stock: {quantite}")
        print('-'*28)

#La fonction pour ajouter un article
def ajouter_article ():
    print("\n**** Ajouter un article ****")
    print("-" * 28)
    nom_product = ''
    while not nom_product:
        nom_product = input("Entrez le nom de l'article à ajouter: ").strip()
        if not nom_product:
            print("Veuillez entrer un nom de produit valide !") #entré vide!
            continue
    inventaire_lowercase = {key.lower(): key for key in inventaire}
    if nom_product.lower() not in inventaire_lowercase:
        while True:
            quantity = input(f"Entrez la nouvelle quantité pour '{nom_product}': ").strip()
            if quantity.isdigit():
                inventaire[nom_product] = int(quantity)
                print(f"Produit ajouté avec succès! '{nom_product}' | Quantité: {quantity}")
                break
            else:
                print("Entrée invalide. Veuillez entrer un nombre.")
                continue
    else:
        print("Article déjà exist !")

```

```

#La fonction pour mettre à jour un article
def update_article ():
    print("\n*** Mettre à jour d'un article ***")
    print("-" * 34)
    inventaire_lowercase = {key.lower(): key for key in inventaire}
    nom_product = input("Entrez le nom de l'article à mettre à jour: ").lower().strip()

    if nom_product in inventaire_lowercase:
        nom_original = inventaire_lowercase[nom_product]
        while True:
            quantity = input("Entrez la nouvelle quantité: ").strip()
            if quantity.isdigit():
                inventaire[nom_original] = int(quantity)
                print(f"Quantité mis à jour pour {nom_original}! (Quantité: {quantity})")
                break
            else:
                print("Entrée invalide. Veuillez entrer un nombre.")
                continue
    else:
        print("Article not exist !")

#La fonction pour supprimer un article
def supprimer_article ():
    print("\n*** Supprimer un article ***")
    print("-" * 28)
    inventaire_lowercase = {key.lower(): key for key in inventaire}
    nom_produit = input("Entrez le nom de l'article à Supprimer : ").lower().strip()
    if nom_produit in inventaire_lowercase:
        nom_original = inventaire_lowercase[nom_produit]
        confirmation_quit = input(f"Voulez vous vraiment supprimer l'article '{nom_original}' ? (Y/N) ")
        if confirmation_quit.lower() == "y":
            del inventaire[nom_original]
            print(f"Article '{nom_original}' supprimé.")
        else:
            print("Article non trouvé !")

#La fonction pour Rechercher un article
def rechercher_article():
    print("\n**** Rechercher un article ****")
    print("-" * 30)
    inventaire_lowercase = {key.lower(): key for key in inventaire}

    nom_rechercher = input("Nom de l'article à rechercher : ").lower().strip()
    if nom_rechercher in inventaire_lowercase:
        nom_original = inventaire_lowercase[nom_rechercher]
        print("-" * 30)
        print(f" '{nom_original}' | Quantité: {inventaire[nom_original]} unités.")
        print("-" * 30)
    else:
        print(f"'{nom_rechercher}' n'est pas dans l'inventaire !")

#Afficher le Rapport Stock faible
def rapport_stock_faible ():
    print("\n**** Rapport de Stock Faible ****")
    print("-" * 34)
    while True:
        seuil = input("Entrez le seuil de Stock faible: ").strip()
        if seuil.isdigit():
            seuil = int(seuil)
            break
        else:
            print("Entrée invalide. Veuillez entrer un nombre.")
    stock_faible = {key: value for key,value in inventaire.items() if value <= seuil}
    print(f"\n•• Stock faible : {len(stock_faible)} articles trouvés. ••")
    print("-" * 40)
    if not stock_faible:
        print("Aucun article avec stock faible !")
    else:
        for article, quantite in stock_faible.items():
            print(f"{article:<22} | Quantité: {quantite}")
        print("-" * 40)

# -- Lancement de Menu --
afficher_menu()

```


Objectif : Extraire des informations précises à partir des logs pour l'optimisation et le débogage.

Exigences principales :

- Comptage des événements par niveau (INFO, WARNING, ERROR).
- Analyse temporelle (événements par heure).
- Liste de messages d'erreur uniques.



Approche : Parsing de chaque ligne de log, utilisation de dictionnaires pour l'agrégation et d'un set pour dédupliquer les erreurs. Retour d'une structure synthétique.

Valeur ajoutée : Outil de monitoring efficace pour identifier rapidement problèmes et pics d'activité.

```
#Rapport d'Analyse de Log Complet
```

```
log_data = """
2025-09-18 09:15:23 [INFO] - L'utilisateur 'admin' s'est connecté.
2025-09-18 09:22:41 [ERROR] - Échec de la connexion à la base de données.
2025-09-18 09:23:15 [WARNING] - Le temps de réponse du disque est élevé.
2025-09-18 10:05:00 [INFO] - Sauvegarde nocturne terminée.
2025-09-18 10:11:34 [ERROR] - Expiration de l'authentification de l'utilisateur.
"""
```

```
def analyser_fichier_log(donnees):
    comptes_par_niveau = {}
    comptes_horaires = {}
    erreurs_uniques = set()

    lignes = donnees.strip().split("\n")

    for ligne in lignes:
        # Découper horodatage et reste
        horodatage, reste = ligne.split(" ", 1)
        heure = horodatage[11:13] # ex: "09"

        # Découper niveau et message
        niveau, message = reste.split("] - ", 1)
        niveau = niveau.strip()

        # Mettre à jour les structures
        comptes_par_niveau[niveau] = comptes_par_niveau.get(niveau, 0) + 1
        comptes_horaires[heure] = comptes_horaires.get(heure, 0) + 1
        if niveau == "ERROR":
            erreurs_uniques.add(message)

    return {
        "comptes_par_niveau": comptes_par_niveau,
        "comptes_horaires": comptes_horaires,
        "erreurs_uniques": erreurs_uniques
    }
```

```
# ----- Utilisation -----
```

```
resultats = analyser_fichier_log(log_data)

print("==== Rapport d'Analyse de Log Complet ==== \n")

print("--- Comptes par Niveau de Log ---")
for niveau, count in resultats["comptes_par_niveau"].items():
    print(f"{niveau}: {count}")

print("\n--- Événements par Heure ---")
for heure, count in resultats["comptes_horaires"].items():
    print(f"Heure {heure}: {count} événements")

print("\n--- Messages d'Erreur Uniques ---")
for err in resultats["erreurs_uniques"]:
    print(f"- {err}")

print("=====")
```


Objectif : Proposer une gestion complète des finances personnelles avec analyse des dépenses.

Exigences principales :

- Ajout, affichage et filtrage des transactions.
- Validation stricte des entrées (type, montant).
- Rapport de synthèse incluant pourcentages par catégorie.



Approche : Stockage des transactions dans une liste, agrégation des données via une fonction de calcul, et affichage formaté via une fonction distincte.

Valeur ajoutée : Vision claire des revenus et dépenses, meilleure gestion financière et prise de décision éclairée.

```

#Outil de gestion de budget
transactions = []

#Fonction main pour afficher le menu principal
def main():
    while True:
        input(f"\nAppuyez sur une touche pour Afficher le Menu Principal.. ")
        print("\n===== Menu Budget =====")
        print("1. Ajouter une transaction")
        print("2. Voir toutes les transactions")
        print("3. Voir transactions par catégorie")
        print("4. Imprimer rapport récapitulatif")
        print("5. Quitter")
        print("-"*28)
        choix = input(f"• Entrez Votre choix : ").strip()

        if not choix.isdigit():
            print("Entrée invalide ! Veuillez entrer un nombre.")
            continue
        #convertir en nombre entier
        choix = int(choix) #convertir en chiffre entier

        if choix == 1:
            ajouter_transaction()
        elif choix == 2:
            voir_toutes_transactions()
        elif choix == 3:
            voir_transactions_par_cat()
        elif choix == 4:
            resume = calculer_resume()
            imprimer_rapport(resume)
        elif choix == 5:
            #confirmation avant Quitter
            confirmation_quit = input(f"Voulez vous vraiment quitter le programme ? (Y/N) ")
            if confirmation_quit.lower() == "n":
                continue
            elif confirmation_quit.lower() == "y":
                print("Au revoir !")
                break
        else:
            print("Choix invalide, veuillez réessayer.")

#fonction ajout de nouvelle transaction
def ajouter_transaction():
    print("\n*** Ajouter transaction ***")
    print('-'*28)
    #Type transaction 'revenu' ou 'depense'
    while True:
        type_entree = input("Entrer le Type 'revenu' ou 'depense' ? : ").strip().lower()
        if type_entree in ("revenu", "depense"):
            break
        print(f"Entrée invalide! Tapez 'revenu' ou 'depense'.")

    #Montant de transaction
    while True:
        montant_entree = input("Entrer le montant de la transaction (ex: 123.45) : ").strip()
        montant_transaction = montant_entree.replace(' ', '').replace(',','.')
        try:
            montant = float(montant_transaction)
            if montant > 0:
                break # valide, on sort de la boucle
            else:
                print("Montant invalide. Entrez un nombre positif et non nul.")
        except ValueError:
            print("Montant invalide. Veuillez entrer un montant valide.")

    #Categorie
    while True:
        categorie = input("Catégorie (ex: Loyer, Nourriture) : ").strip()
        if categorie:
            break
        print(f"Veuillez saisir la catégorie.*")

    description = input("Description (optionnelle) : ").strip()

    transaction = {'type': type_entree,
                   'montant': montant,
                   'categorie': categorie,
                   'description': description}

    transactions.append(transaction)
    print("Transaction ajoutée avec succès.")

#Fonction Affichage de transactions
def voir_toutes_transactions():
    print(f"\n *** {'Voir toutes transactions'.upper()} ***")
    if not transactions:
        print('-' * 34)
        print(f"Aucune transaction enregistrée.")
        return
    #entête
    print('-' * 75)
    print(f"{'Num':<5} {'Type':<20} {'Catégorie':<20} {'montant':<14} {'Description':<40}")
    print('-'*75)
    for i, transaction in enumerate(transactions, 1):
        sign = "-" if transaction['type'] == 'depense' else ""
        montant_formatee = f"{sign}{transaction['montant']:>2f}€"
        print(f"{i:<5} {transaction['type']:<20} {transaction['categorie']:<20} {montant_formatee:<14} {transaction['description']:<40}")

```

```

def voir_transactions_par_cat():
    print("\n*** Voir transactions par catégorie ***")
    if not transactions:
        print('-' * 40)
        print(f"Aucune transaction enregistrée.")
        return

    cat = input("Entrez le nom de la catégorie à filtrer : ").strip().lower()
    filtres = [trans for trans in transactions if trans['categorie'].strip().lower() == cat]

    if not filtres:
        print(f"Aucune transaction trouvée pour la catégorie '{cat}'.")
        return

    print(f"\nTransactions dans la catégorie '{cat}':")
    print('-' * 75)
    print(f"{'Num':<5} {'Type':<20} {'Catégorie':<20} {'montant':<14} {'Description':<40}")
    print('-' * 75)
    for i, transaction in enumerate(filtres, 1):
        sign = "-" if transaction['type'] == 'depense' else ""
        montant_formatee = f"{sign}{transaction['montant']:.2f}€"
        print(f"{i:<5} {transaction['type']:<20} {transaction['categorie']:<20} {montant_formatee:<14} {transaction['description']:<40}")

def calculer_resume():
    total_revenus = 0.0
    total_depenses = 0.0
    depenses_par_categorie = {}

    for trans in transactions:
        if trans['type'] == 'revenu':
            total_revenus += trans['montant']
        else: #depense
            total_depenses += trans['montant']
            cat = trans['categorie'].strip()
            depenses_par_categorie[cat] = depenses_par_categorie.get(cat, 0.0) + trans['montant']

    epargne = total_revenus - total_depenses
    # Pourcentages par catégorie
    pourcentages = {}
    if total_depenses > 0:
        for cat, subtotal in depenses_par_categorie.items():
            pourcentages[cat] = (subtotal / total_depenses) * 100
    else:
        for cat in depenses_par_categorie:
            pourcentages[cat] = 0.0

    resume = {
        'total_revenus': total_revenus,
        'total_depenses': total_depenses,
        'epargne': epargne,
        'depenses_par_categorie': depenses_par_categorie,
        'pourcentages_depenses': pourcentages
    }
    return resume

def imprimer_rapport(resume):
    print("\n===== Résumé Financier =====")
    print(f"{'Revenu Total':<20} {resume['total_revenus']:>10.2f}€")
    # afficher les dépenses comme valeurs négatives pour clarifier
    print(f"{'Dépenses Totales':<20} {-resume['total_depenses']:>10.2f}€")
    print("-" * 33)
    print(f"{'Épargne Nette':<20} {resume['epargne']:>10.2f}€")
    print("\n--- Dépenses par Catégorie ---")
    if not resume['depenses_par_categorie']:
        print("Aucune dépense enregistrée.")
    else:
        # Trier par montant décroissant pour lisibilité
        items = sorted(resume['depenses_par_categorie'].items(), key=lambda x: x[1], reverse=True)
        for cat, montant in items:
            pct = resume['pourcentages_depenses'].get(cat, 0.0)
            print(f"{cat:<12} {montant:>10.2f}€ ({pct:>5.1f}%)")
    print("=" * 32)

#Appel fonction main()
main()

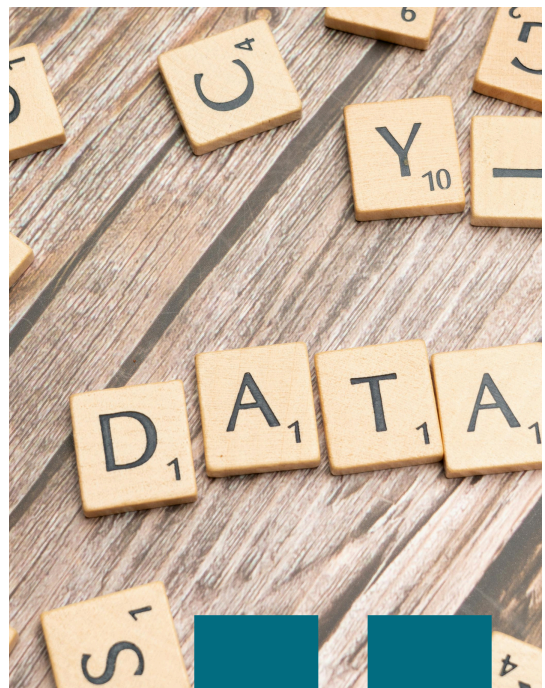
```


CONCLUSION

Ces mini-projets, bien que simples, couvrent des concepts clés de programmation (fonctions, structures de données, validation, parsing, affichage formaté). Ils constituent une base solide pour développer des applications réelles plus complexes et orientées métier.

REMERCIEMENTS

Nous souhaitons exprimer notre gratitude à Monsieur Oussama Ettalali, pour son accompagnement et ses précieux conseils tout au long de la réalisation de ces mini-projets. Nos remerciements vont également à l'équipe JoninTech pour leur soutien technique et collaboratif, ainsi qu'à l'école Ynov Campus qui a fourni le cadre et les ressources nécessaires à la réussite de ce travail.



2025

 Data Makers

JOBINTECH

MERCI

Mohamed - Brahim